CREATING NOVA AI ASSISTANT

A Command-Based CLI System for Advanced Programming

Course Name: Advanced Programming (PCCCS495)
Project Title: Nova AI Assistant (Command-Based CLI System)
Student Name: Swapnoneel Mondal
Roll Number:12024052019068
Instructor Name: Soumadip Biswas
Semester:4th

---

1. Problem Statement

In today's fast-paced digital world, efficiency and speed are paramount when accessing and executing tasks on a computer. Traditional Graphical User Interfaces (GUIs) often come with the overhead of resource consumption and can be cumbersome for experienced users who prefer direct control through text commands. This project, "Nova AI Assistant," serves the need for a lightweight command-line assistant that can perform utility operations, manage tasks, and provide quick outputs without the need for extensive graphical interfaces.

Limitations of Traditional GUI Tools:

- **Resource Intensive:** GUIs typically require more system resources to operate smoothly.
- **Slower Navigation:** Navigating through multiple menus can be time-consuming.
- **Limited Automation:** GUIs often require user interaction for every task, reducing automation potential.

Motivation for a CLI-Based Assistant:

- **Efficiency:** CLI tools can execute commands much faster than GUIs.
- **Automation:** CLI allows for scripting and automation of repetitive tasks.
- **Flexibility:** Offers more flexibility to users who are adept at typing commands.
- **Minimal Resource Usage:** Operates efficiently on systems with limited resources.

---

2. System Design

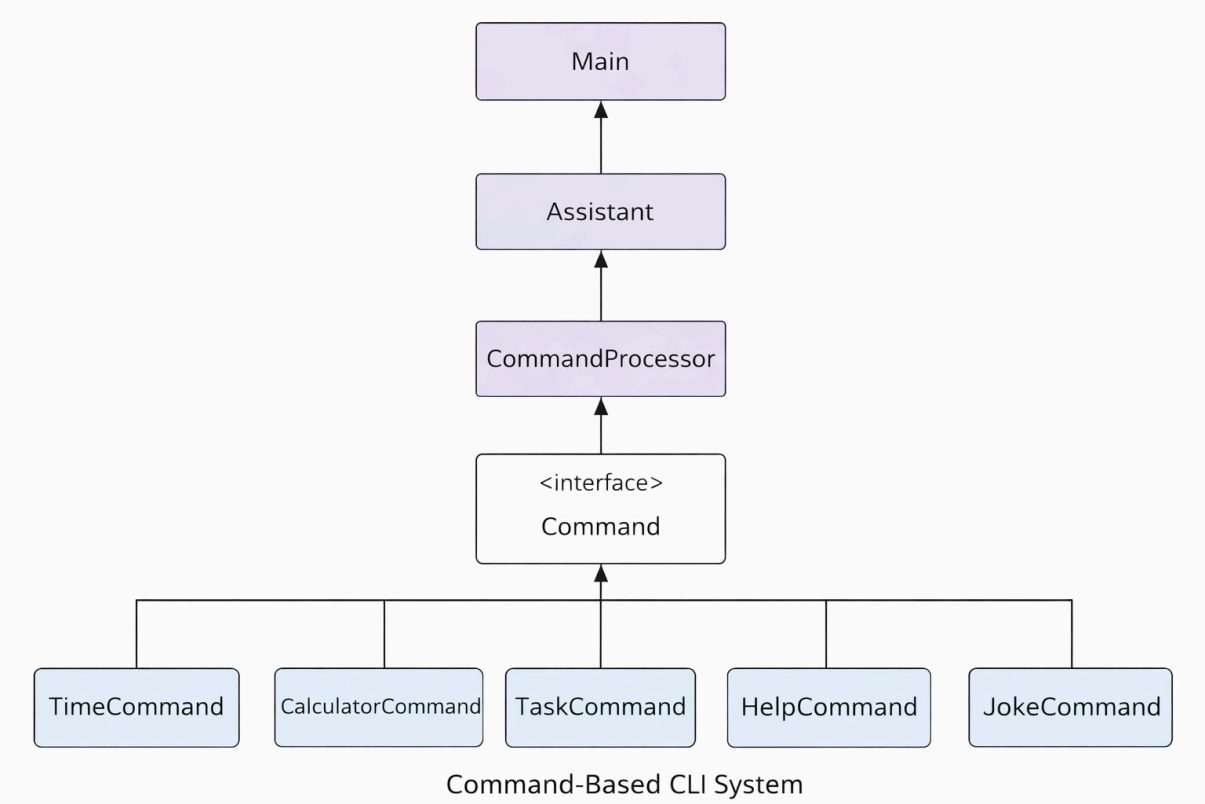
2.1 UML Class Diagram

The UML class diagram for Nova AI Assistant includes the following classes:

- **Main**: Entry point of the application.
- **Assistant**: Central class that interacts with user inputs.
- **Command (interface)**: Defines the structure for all command classes.
- **CommandProcessor**: Processes and executes commands.
- **TimeCommand**: Displays current time.
- **CalculatorCommand**: Performs basic arithmetic operations.
- **TaskCommand**: Manages user-defined tasks.
- **HelpCommand**: Provides help and usage instructions.
- **JokeCommand**: Retrieves and displays jokes.

Relationships:

- **Inheritance**: TimeCommand, CalculatorCommand, TaskCommand, HelpCommand, and JokeCommand inherit from the Command interface.
- **Interface Implementation**: CommandProcessor uses the Command interface to process various commands.



2.2 Description of Major Classes

- **Main**: Initializes the application and handles setup.
- **Assistant**: Acts as the mediator between user inputs and the command processing logic.
- **Command**: Defines a common interface for all command classes, ensuring uniform execution.
- **CommandProcessor**: Maps user inputs to the appropriate command and executes them.
- **TimeCommand**: Provides functionality to fetch and display the current time.
- **CalculatorCommand**: Executes basic arithmetic operations based on user inputs.
- **TaskCommand**: Handles creation, deletion, and listing of tasks.
- **HelpCommand**: Offers guidance on available commands and their usage.
- **JokeCommand**: Fetches and displays a random joke.

2.3 Package Structure

- **core package**: Contains the main logic of the application, including Main, Assistant, and CommandProcessor.
- **commands package**: Contains all implementations of the Command interface, such as TimeCommand, CalculatorCommand, etc.

2.4 Interaction Flow

The flow of interaction in the Nova AI Assistant follows these steps:

1. **User Input**: The user enters a command in the CLI.
2. **Assistant**: The input is received and processed by the Assistant class.
3. **CommandProcessor**: The Assistant forwards the command to the CommandProcessor.
4. **Specific Command**: The CommandProcessor identifies and executes the specific command.
5. **Output**: The result is displayed to the user.

---

3. OOP Concepts Demonstrated

3.1 Encapsulation

Where Used: In the TaskCommand class to manage tasks.

Why Used: To protect the task list from external modification and to provide controlled access through methods.

Short Code Snippet:

```
public class TaskCommand implements Command {
   private List<String> tasks = new ArrayList<>();

   public void addTask(String task) {
      tasks.add(task);
   }

   public List<String> getTasks() {
      return Collections.unmodifiableList(tasks);
   }
```

```
}
```

Explanation: The task list is encapsulated within the class, and only specific methods allow interaction with it, ensuring data integrity.

3.2 Abstraction

Where Used: In the Command interface.

Why Used: To provide a common structure for all commands, allowing them to be processed uniformly.

Short Code Snippet:

```
public interface Command {
   void execute();
}
```

Explanation: The Command interface abstracts the execution details, allowing different commands to be handled in a consistent manner.

3.3 Inheritance

Where Used: In the command implementations like TimeCommand.

Why Used: To inherit the basic structure from the Command interface and add specific functionalities.

Short Code Snippet:

```
public class TimeCommand implements Command {
   @Override
   public void execute() {
      System.out.println("Current time: " + LocalTime.now());
   }
}
```

Explanation: TimeCommand inherits from the Command interface and implements its own version of the execute method.

3.4 Polymorphism

Where Used: In the CommandProcessor class.

Why Used: To allow different command objects to be processed through a common interface.

Short Code Snippet:

```
public class CommandProcessor {
   private Map<String, Command> commandMap;

   public void process(String commandName) {
      Command command = commandMap.get(commandName);
      if (command != null) {
         command.execute();
      }
   }
}
```

Explanation: The process method can execute any command that implements the Command interface, showcasing polymorphism.

3.5 Interface Usage (Command Pattern)

Where Used: Throughout the application for command execution.

Why Used: To decouple the command execution logic from the user input handling.

Short Code Snippet:

```
public class Assistant {
   private CommandProcessor commandProcessor;

   public void handleInput(String input) {
      commandProcessor.process(input);
   }
}
```

Explanation: The Command interface allows the Assistant class to handle inputs without needing to know the specifics of each command.

---

4. Implementation Highlights

4.1 Command Interface

Snippet:

```
public interface Command {
   void execute();
}
```

Explanation: This interface provides a blueprint for all command classes, ensuring they implement the execute method.

4.2 CommandProcessor Mapping

Snippet:

```
public class CommandProcessor {
   private Map<String, Command> commandMap = new HashMap<>();

   public void registerCommand(String name, Command command) {
```

```
        commandMap.put(name, command);
    }
}
```

Explanation: This snippet shows how commands are mapped, enabling dynamic execution based on user input.

4.3 Task Management Logic

Snippet:

```
public class TaskCommand implements Command {
    private List<String> tasks = new ArrayList<>();

    public void addTask(String task) {
        tasks.add(task);
    }

    public void listTasks() {
        tasks.forEach(System.out::println);
    }
}
```

Explanation: This code demonstrates the ability to add and list tasks, providing core task management functionality.

4.4 Input Handling in Assistant

Snippet:

```
public class Assistant {
    private CommandProcessor commandProcessor;

    public void handleInput(String input) {
        commandProcessor.process(input);
    }
}
```

Explanation: The handleInput method processes user inputs, demonstrating the separation of concerns between input handling and command execution.

4.5 Output Demonstration

HELP -

```
Command Prompt    Windows PowerShell

PS C:\Users\Swapnoneel\desktop\NovaAI\src> java Main
Nova AI Assistant Started (type 'exit' to quit)
You: hi
Nova: Hey there! ? How can I help?
You: help

=========== NOVA AI HELP ===========

?? Basic Commands:
  time                ? Show current time
  calculate a + b     ? Perform calculation

? Tasks:
  task add <text>     ? Add a task
  task list           ? Show all tasks
  task remove <n>     ? Remove a task

? Fun:
  joke                ? Get a random joke

? System:
  history             ? Show command history
  help                ? Show this menu
  exit                ? Quit

====================================

You: time
Nova: Current time is 23:24:32.547499200
You:
```

CALCULATE -

```
You: calculate 100 * 99
Nova: Result = 9900.0
```

TASK LIST -

```
You: task list
Nova: Your Tasks:
1. study
```

TASK -

```
You: task add study
Nova: Task added ?
```

---

5. Testing & Error Handling

5.1 Test Cases

- **Valid Commands:** Execute commands like time, add task, calculate 2+2.
- **Invalid Commands:** Test with unrecognized commands like xyz.
- **Edge Cases:** Handle empty inputs and incorrect syntax gracefully.

5.2 Failure Handling

- **Unknown Command:** Provide a default response indicating the command isn't recognized.
- **Incorrect Parameters:** Prompt the user to correct their input with appropriate usage instructions.

---

6. Git Workflow

Use of Git for Version Control:
Git was employed to manage version control throughout the development of the Nova AI Assistant. Multiple commits were made consistently to track progress and facilitate collaboration when needed.

Development Process:

- **Initial Setup:** Basic structure and initial commands were established.
- **Adding Commands Step-by-Step:** Each command was developed and tested incrementally.
- **Final Working Version:** After thorough testing, the final version was committed.

---

7. Conclusion & Future Scope

7.1 Conclusion

The Nova AI Assistant project successfully demonstrated the implementation of a command-based CLI system that is efficient, modular, and user-friendly. The design highlights encapsulation, inheritance, and polymorphism, making the system versatile and easily extensible.

7.2 Future Scope

- **GUI Integration:** Develop a graphical interface for users who prefer visual interactions.
- **Voice Input:** Implement voice recognition for hands-free operation.
- **API Integration:** Incorporate APIs for fetching data from external sources.
- **Advanced AI Features:** Explore machine learning capabilities to enhance command suggestions and user assistance.

This project laid the groundwork for advanced features that can be built upon to create a robust and comprehensive AI assistant.